CONFIDENTIAL • MARCH 2026

Card Network

ISO 8583 Payment Network — Capability Overview

Real-time authorization · Fraud detection · Interchange pricing · Clearing & Settlement

4	5	7	70
Message Types	Fraud Rules	Fee Rate Rules	Tests Passing

OVERVIEW

What Is This?

A fully functional, production-grade **ISO 8583 payment switch** — the same protocol used by Visa, Mastercard, and every major card network. It accepts TCP connections from acquirer banks, routes authorization requests to issuer banks, applies real-time fraud scoring, runs end-of-day clearing and settlement, and calculates **interchange and network fees** per transaction.

Real-time Switch

Netty TCP server on port 8583 handles concurrent ISO 8583 connections. Sub-millisecond routing for 200+ simultaneous transactions.

Fraud Engine

5 configurable fraud rules with score-based aggregation. Decisions in <1ms, before any issuer contact.

Interchange Pricing

Per-transaction interchange and network fees calculated at clearing. MCC-based rate lookup, configurable schedule, applied to net settlement.

Clearing & Settlement

Daily batch clearing at 23:00 UTC, net position settlement at 23:30 UTC. Net position includes interchange and network fee adjustments.

Operations Dashboard

Browser-based admin UI with live transaction monitoring, fraud alert review queue, fee schedule management, and manual clearing controls.

AWS-Ready

Terraform IaC for ECS Fargate multi-AZ deployment. Aurora PostgreSQL, ALB + NLB, ShedLock for distributed jobs.

ISO 8583 Transaction Flow

AUTHORIZATION (0100 → 0110)

Acquirer Bank → **0100 Request** (PAN · Amount · MCC · STAN) → **Fraud Engine** (Score · Decide) → **BIN Router** (PAN → Issuer) → **Issuer Bank** (0100 forwarded) → **0110 Response** (RC · Auth ID) →

Acquirer Bank

END-OF-DAY PROCESSING

Approved Txns → **ClearingEngine** (batch records) → **InterchangeService** (MCC → rate → fee) → **SettlementEngine** (net positions) → **SettlementRecords** (with fee breakdown)

0100 / 0110	0200 / 0210	0400 / 0420	0800 / 0810
Authorization	Financial	Reversal	Network Mgmt
Request & Response	Request & Response	Request & Advice	Sign-on · Echo
implemented	implemented	implemented	implemented

KEY ISO 8583 DATA ELEMENTS

- **DE2** — Primary Account Number (PAN)
- **DE3** — Processing Code
- **DE4** — Transaction Amount
- **DE11** — STAN (trace number)
- **DE18** — MCC → fee rate lookup
- **DE32/33** — Institution Codes
- **DE37** — Retrieval Reference Number
- **DE38** — Authorization ID
- **DE39** — Response Code
- **DE41** — Terminal ID
- **DE49** — Currency Code
- **DE70** — NMC (Network Mgmt)

FRAUD DETECTION

Real-time Fraud Engine

HOW IT WORKS

Every transaction is scored **before** it reaches the issuer. Each enabled rule contributes a weight to the aggregate score (0–100).

Score Range	Action	RC
≥ 80	DECLINE	59 — Suspected Fraud
≥ 50	FLAG	Continue to issuer + alert
< 50	APPROVE	Continue normally

5 RULE IMPLEMENTATIONS

— **Velocity** — Max N transactions per card per hour

— **Amount Limit** — Flags amounts above configurable threshold ($5k default)

— **Blacklist** — Instant DECLINE for blacklisted PANs (SHA-256 hashed)

— **Decline Velocity** — Detects card-testing: ≥3 declines/hour

— **MCC Risk** — Flags high-risk merchant categories (gambling, pawnshops, crypto)

FRAUD OPERATIONS

DB-Driven Config

Rules stored in PostgreSQL with JSON parameters. Enable/disable and adjust thresholds without redeployment. Cache refreshes every 60s.

Alert Queue

Every flagged or declined transaction creates a FraudAlert record. Dashboard shows a review queue with fraud score and triggered rule names.

PAN Blacklist

PANs added as SHA-256 hashes — raw card numbers are never stored. Supports optional expiry timestamps for temporary blocks.

CLEARING & SETTLEMENT

End-of-Day Processing

DAILY SCHEDULE

Time (UTC)	Process	Output
23:00	Clearing	ClearingBatch · ClearingRecords with fees per txn
23:30	Settlement	Net position per participant including fee adjustments

ShedLock (Multi-Instance)

PostgreSQL-backed distributed lock ensures only one ECS task executes clearing or settlement at a time across any number of instances.

SETTLEMENT NET POSITION FORMULA

```
credit = Σ amounts where participant = issuer
debit = Σ amounts where participant = acquirer

iReceived = Σ interchange received (as issuer)
iPaid = Σ interchange paid (as acquirer)
nFees = Σ network fees paid (both sides)

net = credit − debit + iReceived − iPaid − nFees
```

- Issuers gain interchange income from every transaction they issue
- Acquirers pay interchange on every transaction they process
- Both sides pay a scheme (network) fee per transaction
- One SettlementRecord per participant — stores full fee breakdown

SUPPORTED CONTROLS

Manual Trigger

Dashboard "Run Clearing" button and `POST /clearing/run` endpoint allow out-of-schedule runs for testing.

Idempotency

Clearing is idempotent per date. Re-running for a date with a COMPLETE batch is a safe no-op.

CSV Export

`GET /settlement/{date}/report` and `GET /api/interchange/report?format=csv` for full fee breakdown.

Interchange & Network Fee Engine

HOW FEES ARE CALCULATED

```
fee = round(amount × bps / 10,000) + flat

Example: $100.00, MCC 5411 (retail rate)
interchange = round(10000 × 120 / 10000) + 8 = 128 ¢ ($1.28)
network fee = round(10000 × 5 / 10000) + 2 = 7 ¢ ($0.07)

Example: $100.00, MCC 7995 (high-risk)
interchange = round(10000 × 250 / 10000) + 15 = 265 ¢ ($2.65)
```

Rate Matching Logic

1. Look up exact MCC in `interchange_rates` (highest priority wins)
2. Fall back to DEFAULT catch-all rate if no exact match
3. Return 0 if no rate configured — never blocks clearing

TWO FEE CATEGORIES

Category	Direction	Applied To
INTERCHANGE	Acquirer → Issuer	Every cleared transaction
NETWORK	Both → Network	Both sides pay per transaction

MCC Persistence

DE18 (Merchant Category Code) is captured from every inbound ISO 8583 message and stored on the Transaction record. At clearing time, InterchangeService uses the MCC for rate lookup.

DEFAULT RATE SCHEDULE (7 SEEDED RULES)

Name	Category	MCC Pattern	Rate (bps)	Flat	Priority	Description
Interchange Standard	INTERCHANGE	DEFAULT	150 bps (1.50%)	$0.10	0	Catch-all rate for unmatched MCCs
Interchange Retail	INTERCHANGE	5411	120 bps (1.20%)	$0.08	5	Grocery & retail — preferred rate
Interchange High-Risk	INTERCHANGE	7995	250 bps (2.50%)	$0.15	10	Gambling / betting
Interchange High-Risk	INTERCHANGE	5933	250 bps (2.50%)	$0.15	10	Pawnshops
Interchange High-Risk	INTERCHANGE	6051	250 bps (2.50%)	$0.15	10	Crypto & non-bank financial
Network Fee — Acquirer	NETWORK	DEFAULT	5 bps (0.05%)	$0.02	0	Scheme fee — acquirer side

MANAGEMENT INTERFACES

REST API & Admin Dashboard

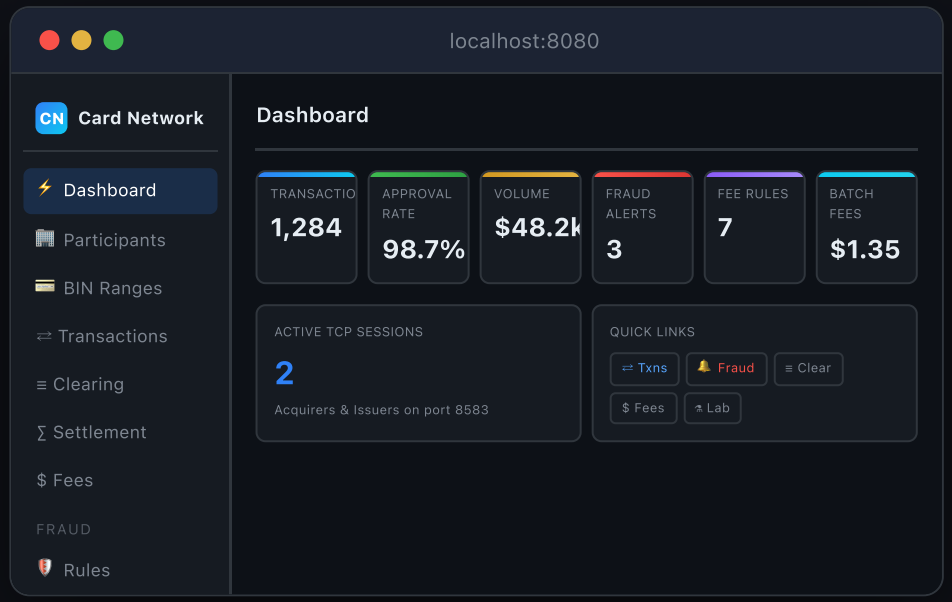
REST API (HTTP BASIC AUTH)

Domain	Operations
Participants	CRUD — create, read, update, deactivate
BIN Ranges	List, create, delete routing entries
Transactions	Paginated list, detail view
Fraud Rules	Full CRUD with live JSON parameter editing
Fraud Alerts	Paginated list, mark reviewed
Blacklist	Add (raw PAN → hash), list, remove
Clearing	List batches, trigger run
Settlement	List records, CSV export with fee columns
Interchange	CRUD rates · Fee report (JSON + CSV)
Test Lab	List scenarios, run, view results

Security Model

Dashboard: Spring Security form login · REST: HTTP Basic
CSRF disabled for `/api/**` · PANs masked (first 6 + last 4)

DASHBOARD UI (DARK THEME · BOOTSTRAP ICONS)



Dark theme (GitHub palette) · Bootstrap Icons nav · Gradient accent stat cards · Hover lift animation · 12 pages

Happy Path	End-to-end approval, auth ID generated
Issuer Decline	RC=05 propagated correctly to acquirer
BIN Not Found	RC=15 when no BIN range matches
Blacklisted Card	Fraud DECLINE (RC=59), alert created
High-Risk MCC	MCC=7995 → fraud FLAG, issuer approves
Large Amount	Amount > $5k → fraud FLAG
Velocity Breach	7 rapid txns trigger velocity alert
Decline Velocity	4 declines → DECLINE (RC=59)
Echo Test	0800 NMC=301 → 0810 RC=00 immediately
Reversal	Approve then 0400 → 0420 RC=00, status REVERSED
Volume Load	**200 sequential transactions**, all approved
Mixed Batch	8 txns: happy + BIN miss + fraud + large amount

— **InterchangeServiceTest** — MCC lookup, DEFAULT fallback, zero return

— **SettlementEngineTest** — Fee aggregation, net position formula

— **TransactionLoggerTest** — MCC captured from DE18

— **InterchangeControllerTest** — CRUD + CSV export

— **FeesIntegrationTest** — Flyway seeds 7 rates; end-to-end fee math

— **ClearingEngineTest** — Fees on records, totalFees on batch, MCC routing

— **MessageFactoryTest** — ISO 8583 encode/decode

— **BinRouterTest** — BIN range lookup edge cases

— **FraudDetectionServiceTest** — Rule scoring, thresholds

— **GatewayIntegrationTest** — Full TCP stack, Testcontainers PG

— **AuthorizationE2ETest** — Full auth flow: acquire → fraud → route → issue → respond

— **LiveAuthDemoTest** — Real-time 0100 approval with human-readable log output

— **VolumeTest** — 3 scenarios: 500 concurrent txns, 200-burst, 100 connection churn cycles

— **DurationTest** — 2 scenarios: 30-second sustained (≥60 txns), 5-client 20-second (≥100 txns)

Infrastructure

Testcontainers spins up real PostgreSQL with all Flyway migrations applied. Interchange rate seeds verified at integration level. ISO 8583 bytes over TCP. Colima (macOS Docker) supported.

Bug Found & Fixed in Production Validation

Browser testing of the new `/fees` page revealed a Thymeleaf template

PERFORMANCE

Volume & Duration Tests — 5 Tests, 0 Failures

VOLUMETEST — CONCURRENT LOAD

Scenario	Load	Pass Threshold
Concurrent 500	10 clients × 50 txns	≥ 490 approved (98%)
Burst 200	20 clients × 10 txns	≥ 196 approved (98%)
Connection Churn	100 sequential sign-on / off cycles	100 / 100 complete

DURATIONTEST — SUSTAINED THROUGHPUT

Scenario	Duration	Pass Threshold
Sustained 30 s	1 client, back-to-back, 30 seconds	≥ 60 txns · 0 failures
Multi-client 20 s	5 clients simultaneously, 20 seconds	≥ 100 txns (≥ 10 tps)

Race Condition — Found & Fixed

Writing concurrent tests immediately exposed a race in `ParticipantConnector.send()`: multiple threads each saw an empty session registry and called `bootstrap.connect().sync()` simultaneously. Only 1 connection was accepted by the fake issuer's `ServerSocket`; the remaining N−1 were zombie channels. Last `register()` overwrote the registry with a zombie → only 1 transaction approved, all others timed out.

Fix: Per-issuer double-checked locking with `ConcurrentHashMap<String, Object>` lock objects. The check-then-connect sequence is now atomic per issuer code. All 5 performance tests pass. Run time dropped from 17 min → 2:40 min.

Test Infrastructure

Each class runs a full Spring context with Testcontainers PostgreSQL. A raw Java `ServerSocket` plays the issuer, parsing ISO 8583 frames and replying with 0110 approvals. Participants sign on with 0800 NMC=001 before the test loop. Unique gateway ports (18590 / 18591) prevent inter-class interference.

Outcome	Condition	Dispute Status
ACCEPTED	Issuer responds RC=00	ACCEPTED
REJECTED_BY_ISSUER	Issuer responds RC≠00	REJECTED_BY_ISSUER
TIMEOUT	No 0210 within timeout window	RC=91 → PENDING

DATABASE — V11 FLYWAY MIGRATION

```
CREATE TABLE disputes (
  id BIGSERIAL PRIMARY KEY,
  transaction_id BIGINT REFERENCES transactions(id),
  claimant_id BIGINT REFERENCES participants(id),
  reason_code VARCHAR(10) NOT NULL,
  status VARCHAR(30) NOT NULL,
  amount BIGINT NOT NULL,
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
```

↓

ARBITRATION

↓

WON | LOST | WITHDRAWN

ADMIN DASHBOARD & REST API

Dashboard UI

- `GET /disputes` — list with status filter dropdown & pagination
- `GET /disputes/{id}` — lifecycle detail view with action buttons
- Stat card on main dashboard: **Open Disputes** count with purple accent

REST API (HTTP Basic)

- GET `/api/disputes` — paginated list, filter by status
- POST `/api/disputes` — file new chargeback
- PUT `/api/disputes/{id}` — advance lifecycle state
- GET `/api/disputes/stats` — count by status, total disputed amount

REASON CODE REFERENCE

VISA REASON CODES

Code	Category	Description
10.1	EMV Liability	Counterfeit — chip card used at mag-stripe

MASTERCARD REASON CODES

Code	Category	Description
4808	Authorization	Required authorization not obtained

ARCHITECTURE

System Architecture

PARTICIPANTS

Acquirer Banks
TCP connect port 8583

Issuer Banks
TCP connect port 8583

TCP GATEWAY (NETTY)

GatewayServer
2-byte length-prefix framing · jPOS packager · SessionRegistry

GatewayMessageHandler
Routes by MTI → Auth / Financial / Reversal / NMC service

BUSINESS LOGIC

FraudDetectionService
5 rules · Score aggregation · DB-cached config

BinRouter
PAN → BinRange → Issuer

InterchangeService
MCC → rate lookup · INTERCHANGE + NETWORK fees per txn

ClearingEngine · SettlementEngine
Daily batch jobs · ShedLock · fee-adjusted net positions

PERSISTENCE

PostgreSQL 16
10 migrations · 11 entities · interchange_rates table

Spring Data JPA
11 repositories · Hikari pool

TECHNOLOGY STACK

Java 21 + Spring Boot 3.4
Virtual threads ready, modern Spring ecosystem

jPOS 2.1.9
ISO 8583 encode/decode via xml packager config

Netty 4.1
Non-blocking TCP gateway, async handlers

Flyway 11
10 versioned migrations, schema owned by Flyway

GAPS & ROADMAP

What's Still Missing

CORE PROTOCOL GAPS

- **Duplicate / Retransmission Detection**
  No idempotency on STAN. A retransmitted 0100 with the same STAN would create a duplicate transaction.

- **Batch File Support (ISO 8583 / ACH)**
  Only real-time TCP. No file-based clearing upload or bulk authorization advices.

- **3D Secure / PSD2 SCA**
  No authentication challenge flow (DE55 / EMV data, liability shift logic).

- **Tiered / Qualified Interchange**
  Real networks apply multi-tier rates (qualified / non-qualified / rewards). Current model uses a single rate per MCC with no card-type differentiation.

OPERATIONAL GAPS

- **Role-Based Access Control (RBAC)**
  Single admin credential. No read-only analyst role, no per-participant portal login, no API key management.

- **Audit Logging**
  Fraud rule changes, participant config edits, interchange rate changes, and blacklist additions are not persisted to an audit trail.

- **Real-time Reporting & Charts**
  Dashboard shows daily totals only. No hourly charts, trend lines, fraud rate over time, or fee revenue trends.

- **Distributed Tracing (OTEL / Zipkin)**
  No trace IDs propagated through multi-hop flows. Debugging a slow issuer requires log correlation by STAN.

- **Dead-Letter Queue & Retry**
  Issuer timeouts respond immediately with RC=91. No retry queue or delayed re-delivery for transient failures.

SUMMARY

What We Have vs. What's Next

BUILT & WORKING

- Full ISO 8583 switch (0100, 0200, 0400, 0800)
- Real-time fraud engine — 5 configurable rules
- BIN routing — PAN → issuer bank
- Interchange & network fee calculation — 7 seeded rates
- Clearing & settlement with fee-adjusted net positions
- Fee schedule management & per-participant fee report
- Chargeback & dispute handling — full lifecycle, reason codes, REST API
- Operations dashboard — 12 pages
- REST API — 35+ endpoints
- Interactive Test Lab — 12 scenarios
- 70 automated tests (unit + integration + volume + duration)
- Terraform IaC for AWS ECS Fargate multi-AZ

NEXT PRIORITIES

- Duplicate / retransmission detection on STAN
- Tiered interchange (qualified / non-qualified / rewards)
- Role-based access control
- Audit trail for config changes
- Real-time charts & trend reporting
- 3D Secure / EMV authentication data
- Distributed tracing (OTEL)
- HTTPS + ACM cert (AWS deployment)

4	5	7	70
Message Types	Fraud Rules	Fee Rate Rules	Tests Passing